

Examen Final

Programación 1 (F12) — ECFM / USAC

Semestre I — 2026

El examen final tiene **dos partes**: un análisis exploratorio de datos con `pandas/NumPy/seaborn` (Ejercicios 1–4) y un ejercicio de **Programación Orientada a Objetos** en Python (Ejercicio 5). Hay **dos variantes**; puedes elegir la que prefieras.

	Variante A	Variante B
Tema	Sismos (USGS)	Clima (Open-Meteo)
Datos	CSV directo	JSON → DataFrame
Período	Ene–Jun 2024	Todo 2024
Región	Global	Ciudad de Guatemala
Notebook EDA	<code>examen_final_a_estudiante.ipynb</code>	<code>examen_final_b_estudiante.ipynb</code>
Script POO	<code>poo_sismos.py</code>	<code>poo_clima.py</code>
Notebook POO	<code>examen_final_poo_a.ipynb</code>	<code>examen_final_poo_b.ipynb</code>

El enunciado completo y el notebook de tu variante están disponibles en la **U Virtual**, en la misma sección que este documento.

Variante A — Análisis de Datos Sísmicos

Variante A — Análisis de Datos Sísmicos

Fuente: United States Geological Survey (USGS) — earthquake.usgs.gov

El dataset contiene todos los sismos de magnitud $\geq 5,5$ registrados a nivel global durante el primer semestre de 2024 (enero–junio). El USGS publica estos datos de forma abierta; la API entrega un CSV directo, **sin API key**.

Cómo cargar los datos (ya incluido en el notebook):

```

1 import pandas as pd
2
3 URL_SISMOS = (
4     "https://earthquake.usgs.gov/fdsnws/event/1/query"
5     "?format=csv&starttime=2024-01-01&endtime=2024-06-30"
6     "&minmagnitude=5.5"
7 )
8
9 sismos = pd.read_csv(URL_SISMOS)
```

Columnas principales: time, latitude, longitude, depth (km), mag, magType, place.

Ejercicios y punteo:

Ejercicio	Tema	Pts
1a	Cargar datos con <code>pd.read_csv()</code> desde URL	10
1b	Explorar el DataFrame con un ciclo <code>for</code> e <code>iterrows()</code>	5
2a	Encontrar el sismo más intenso (ciclo <code>for</code> + <code>if</code>)	10
2b	Top 5 más profundos con <code>.loc[]</code> + <code>sort_values()</code>	5
3a	Función <code>clasificar_sismo(mag)</code> con condicionales	10
3b	Aplicar la función con <code>.apply()</code> y <code>value_counts()</code>	10
4a	Función <code>analizar_sismos(df)</code> que retorna diccionario	10
4b	Tres visualizaciones (histograma, scatter, barras)	15
<i>Ejercicio 5 — Programación Orientada a Objetos</i>		
5a	Implementar métodos de <code>Sismo</code> : <code>clasificar()</code> , <code>clasificar_profundidad()</code> , <code>es_peligroso()</code> , <code>descripcion()</code>	10
5b	Implementar métodos de <code>CatalogoSismos</code> : <code>el_mas_intenso()</code> , <code>filtrar_por_categoria()</code> , <code>resumen()</code>	10
5c	Ejecutar notebook POO y responder preguntas de interpretación	5
Total		100
<i>Ejercicio extra (opcional)</i>		
2c	Usando un ciclo <code>for</code> , identifica los sismos peligrosos ($\text{mag} \geq 7.0$ y profundidad < 70 km); imprime cuántos hay y cuál es el más intenso entre ellos	+10
Total máximo		110

Variante B — Análisis de Datos Climáticos

Variante B — Análisis de Datos Climáticos

Fuente: Open-Meteo Historical Weather — open-meteo.com

El dataset contiene el registro climático diario de **Ciudad de Guatemala** durante el año completo 2024 (366 días, año bisiesto). Open-Meteo es una API meteorológica de código abierto; **no requiere API key**.

Cómo cargar los datos (ya incluido en el notebook):

```
1 import requests
```

```
2 import pandas as pd
3
4 URL_CLIMA = (
5     "https://archive-api.open-meteo.com/v1/archive"
6     "?latitude=14.6349&longitude=-90.5069"
7     "&start_date=2024-01-01&end_date=2024-12-31"
8     "&daily=temperature_2m_max,temperature_2m_min,"
9     "precipitation_sum,wind_speed_10m_max"
10    "&timezone=America%2FGuatemala"
11 )
12
13 respuesta = requests.get(URL_CLIMA)
14 datos = respuesta.json()
15 clima = pd.DataFrame(datos["daily"])
16 clima["time"] = pd.to_datetime(clima["time"])
```

Columnas principales: time, temperature_2m_max (°C), temperature_2m_min (°C), precipitation_sum (mm), wind_speed_10m_max (km/h).

Ejercicios y punteo:

Ejercicio	Tema	Pts
1a	Cargar datos con <code>requests + pd.DataFrame()</code>	10
1b	Explorar el DataFrame con un ciclo <code>for</code> e <code>iterrows()</code>	5
2a	Encontrar el día más caluroso (ciclo <code>for + if</code>)	10
2b	Top 5 días con más lluvia con <code>.loc[] + sort_values()</code>	5
3a	Función <code>clasificar_dia(precip)</code> con condicionales	10
3b	Aplicar la función con <code>.apply()</code> y <code>value_counts()</code>	10
4a	Función <code>analizar_clima(df)</code> que retorna diccionario	10
4b	Tres visualizaciones (histograma, scatter, barras por mes)	15
<i>Ejercicio 5 — Programación Orientada a Objetos</i>		
5a	Implementar métodos de <code>DiaClimatico</code> : <code>rango_termico()</code> , <code>temp_media()</code> , <code>es_caluroso()</code> , <code>clasificar()</code> , <code>descripcion()</code>	10
5b	Implementar métodos de <code>RegistroAnual</code> : <code>dia_mas_caluroso()</code> , <code>dias_por_tipo()</code> , <code>temp_promedio_anual()</code> , <code>resumen()</code>	10
5c	Ejecutar notebook POO y responder preguntas de interpretación	5
Total		100
<i>Ejercicio extra (opcional)</i>		
2c	Usando un ciclo <code>for</code> , encuentra el mes con mayor precipitación total; imprime el mes, la cantidad de días lluviosos y la precipitación acumulada	+10
Total máximo		110

Recursos disponibles y honestidad académica

Recursos y honestidad académica

El examen está diseñado para resolverse con los recursos del curso. Los notebooks de clase (disponibles en el repositorio de GitHub de la cátedra) contienen ejemplos directamente aplicables a cada ejercicio: `pandas`, `NumPy`, ciclos `for`, funciones con condicionales, `matplotlib/seaborn` y consumo de APIs con `requests`. También puedes consultar la documentación oficial enlazada en los *Hints* del propio notebook.

Confiamos en que dedicarás el tiempo necesario a entender los problemas, consultar los notebooks de la clase y resolver los ejercicios de forma independiente. Ese proceso —equivocarte, corregir, volver a intentarlo— es exactamente lo que te convierte en programador.

Sobre el uso de inteligencia artificial:

- **No uses IA para generar el código del examen.** El objetivo es que *tú* programes y aprendas, no que valides lo que genera una herramienta automática.
- Se verificará que **no existan copias exactas de código** entre entregas, ni patrones consistentes con código generado por IA (nombres de variables genéricos, estructura idéntica entre variantes, comentarios que no reflejan comprensión propia, etc.).
- Las **preguntas de interpretación** y la **conclusión final** requieren valores específicos de tu ejecución: no pueden completarse sin correr el notebook tú mismo.

Instrucciones generales

Antes de empezar

1. Descarga el **archivo comprimido** (.zip) de tu variante desde la **U Virtual**. Contiene: el notebook EDA, el script POO y el notebook POO.
2. **Descomprime** el archivo en una carpeta local de tu computadora.
3. **Crea un nuevo repositorio en GitHub** (público, sin README). Sube los tres archivos a la raíz del repositorio.
4. Abre el notebook EDA en **Jupyter Notebook** o **Google Colab**.
5. Completa tu **NOMBRE** y **CARNET** en la celda de identificación. El notebook lanzará un error si dejas los valores por defecto.
6. Ejecuta las celdas **en orden**, de arriba hacia abajo.
7. Escribe tu código únicamente donde indica **# TU CÓDIGO AQUÍ**.

Sobre los ejercicios

- Los ejercicios 1 y 2 requieren **ciclos for y condicionales if**; no uses `.max()`, `.idxmax()` ni filtros de pandas en esas partes.
- Los ejercicios 3 y 4 sí permiten — y esperan — el uso de pandas y NumPy.
- El **ejercicio 5** es requerido y tiene dos pasos: (1) abre el script `.py` e implementa los métodos marcados con **# TU CÓDIGO AQUÍ**; (2) abre el notebook POO y ejecútalo completo para verificar que tus métodos funcionan correctamente.
- El **ejercicio 2c** es **opcional** (+10 pts extra). Usa un ciclo **for**; no se permite `.query()` ni filtros directos de pandas.
- Cada celda de código incluye **# Hint**: con referencias a la documentación oficial; úsalas si tienes dudas.
- Las preguntas de interpretación requieren que ejecutes el código y anotes los valores exactos que obtuviste. No puedes responderlas sin correr el notebook.

Sello de entrega

Al finalizar el examen, la última celda de código registra la **posición de la Estación Espacial Internacional (ISS)** en el momento exacto en que la ejecutas. Dado que la ISS viaja a $\sim 27,600$ km/h, dos estudiantes que entreguen en momentos distintos obtendrán coordenadas completamente diferentes.

Este sello vincula tu entrega a tu carnet y a la hora real de ejecución. **Es obligatorio ejecutarlo justo antes de guardar el archivo.**

Importante

Si el sello no tiene output (coordenadas impresas), la celda no fue ejecutada. Las entregas sin sello no serán calificadas.

Entrega

1. **Ejecuta el sello de entrega** (última celda del notebook EDA).
2. **Verifica que todas las celdas tienen output.** Usa *Kernel* $\rightarrow$ *Restart & Run All* para confirmarlo.
3. **Guarda todos los archivos** (Ctrl+S en cada notebook y en el script `.py`).
4. **Sube los tres archivos al repositorio que creaste en GitHub.** Deben quedar en la raíz o en la misma carpeta:

```
examen_final_[a|b]_estudiante.ipynb  poo_[sismos|clima].py
                                examen_final_poo_[a|b].ipynb
```

Verifica que los cambios quedaron guardados haciendo *commit* y *push*.

5. **Copia el enlace al repositorio** (no al archivo individual) y pégalo en el campo de entrega de la **U Virtual** dentro del plazo indicado.

Importante

Asegúrate de que tu repositorio es **público** antes de enviar el enlace. Un repositorio privado no puede ser revisado por los catedráticos. La entrega implica que el código es de tu autoría. Entregas con código idéntico a otro estudiante o generado por IA serán anuladas para todos los involucrados.

Criterios de evaluación

Criterio	Valor
Código correcto y funcional (Ejercicios 1–5)	100 pts
Preguntas de interpretación respondidas con valores exactos	incluido en cada ejercicio
Conclusión final (≥ 5 oraciones con datos específicos)	incluido en Ejercicio 4
Sello de entrega ejecutado (ISS)	requisito de entrega
Ejercicio 2c (opcional, ciclo <code>for</code>)	+10 pts

Las preguntas de interpretación **no tienen punteo separado** pero son parte del punteo de cada ejercicio: una respuesta en blanco o genérica indica que el código no fue ejecutado y reduce el punteo de ese ejercicio.